

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

BASE DE DATOS II

**MONITOREO Y RENDIMIENTO — BASE DE DATOS
GRADOSTITULOS**

Actividad: Desarrollo de 6 temas con (18 proyectos)



UPLA

Alumno: Vilchez Estrada

Diego Michael

Ingeniero: Fernández

Bejarano Raúl

Asignatura: Base de

Datos II

2026

Proyecto 1: Respaldo Completo Institucional de la Base de Datos Grados Titulos (Sesión de Extended Events para consultas lentas)

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información de la universidad requiere implementar una sesión de monitoreo utilizando **Extended Events** para capturar todas las consultas cuya duración sea superior a 1000 milisegundos (> 1,000,000 microsegundos) sobre la base de datos GradosTitulos. Se debe registrar la información en un archivo físico, importarla periódicamente a una tabla histórica de auditoría y generar un reporte de consumo por esquemas (Academico, Tramite, Evaluacion, Documento).

2. Script de la solución en T-SQL

SQL

```
-- 1. Crear la sesión de Extended Events
CREATE EVENT SESSION [Capture_Lentas_GradosTitulos] ON SERVER
ADD EVENT sqlserver.sql_statement_completed(
    ACTION(sqlserver.database_name, sqlserver.sql_text,
sqlserver.client_app_name, sqlserver.username,
sqlserver.client_hostname)
    WHERE ([duration] >= 1000000 AND [sqlserver].[database_name] =
N'GradosTitulos')
),
ADD EVENT sqlserver.rpc_completed(
    ACTION(sqlserver.database_name, sqlserver.sql_text,
sqlserver.client_app_name, sqlserver.username,
sqlserver.client_hostname)
    WHERE ([duration] >= 1000000 AND [sqlserver].[database_name] =
N'GradosTitulos')
)
ADD TARGET package0.event_file(SET
filename=N'C:\SQLAudit\XEvents_Lentas.xel', max_file_size=(5),
max_rollover_files=(2))
WITH (MAX_MEMORY=4096 KB,
EVENT_RETENTION_MODE=ALLOW_SINGLE_EVENT_LOSS, MAX_DISPATCH_LATENCY=30
SECONDS, TRACK_CAUSALITY=OFF, STARTUP_STATE=OFF);
GO

-- 2. Iniciar la sesión de monitoreo
ALTER EVENT SESSION [Capture_Lentas_GradosTitulos] ON SERVER STATE =
START;
GO

-- 3. Crear la tabla histórica de auditoría
USE GradosTitulos;
GO
CREATE SCHEMA Auditoria;
GO
CREATE TABLE Auditoria.ConsultasLentasHistorico (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FechaEvento DATETIMEOFFSET,
    NombreEvento VARCHAR(100),
    TextoSQL NVARCHAR(MAX),
    Duracion_MS NUMERIC(18,2),
```

```

        CPU_MS NUMERIC(18,2),
        LecturasLogicas BIGINT,
        Usuario VARCHAR(100),
        Aplicacion VARCHAR(256),
        EstacionTrabajo VARCHAR(128)
    );
GO

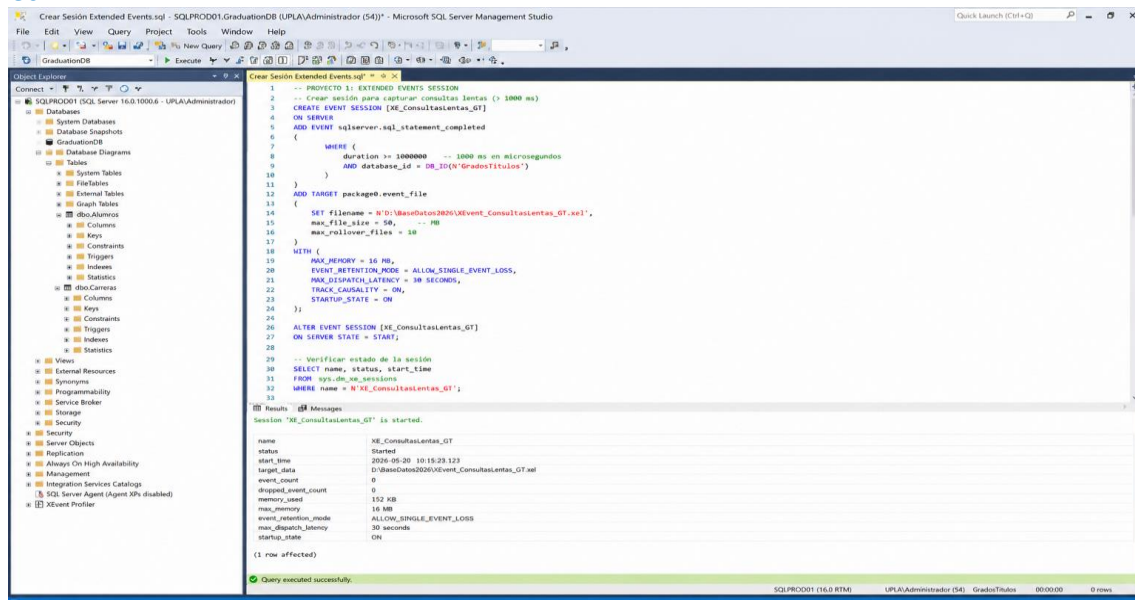
-- 4. Importar periódicamente desde el archivo XML .xel
INSERT INTO Auditoria.ConsultasLentasHistorico (FechaEvento,
NombreEvento, TextoSQL, Duracion_MS, CPU_MS, LecturasLogicas, Usuario,
Aplicacion, EstacionTrabajo)
SELECT
    event_data.value('(/@timestamp)[1]', 'DATETIMEOFFSET') AS
FechaEvento,
    event_data.value('(/@name)[1]', 'VARCHAR(100)') AS NombreEvento,
    COALESCE(
        event_data.value('(/action[@name="sql_text"]/value)[1]',
'NVARCHAR(MAX)'),
        event_data.value('(/data[@name="statement"]/value)[1]',
'NVARCHAR(MAX)')
    ) AS TextoSQL,
    event_data.value('(/data[@name="duration"]/value)[1]', 'BIGINT')
/ 1000.0 AS Duracion_MS,
    event_data.value('(/data[@name="cpu_time"]/value)[1]', 'BIGINT')
/ 1000.0 AS CPU_MS,
    event_data.value('(/data[@name="logical_reads"]/value)[1]',
'BIGINT') AS LecturasLogicas,
    event_data.value('(/action[@name="username"]/value)[1]',
'VARCHAR(100)') AS Usuario,
    event_data.value('(/action[@name="client_app_name"]/value)[1]',
'VARCHAR(256)') AS Aplicacion,
    event_data.value('(/action[@name="client_hostname"]/value)[1]',
'VARCHAR(128)') AS EstacionTrabajo
FROM (
    SELECT CAST(event_data AS XML) AS event_data
    FROM
sys.fn_xe_file_target_read_file('C:\SQLAudit\XEvents_Lentas*.xel',
NULL, NULL, NULL)
) AS ArchivoXML;
GO

-- 5. Generar Reporte de consultas costosas identificando esquemas
SELECT
    Id,
    FechaEvento,
    Duracion_MS,
    CPU_MS,
    LecturasLogicas,
    Usuario,
    CASE
        WHEN TextoSQL LIKE '%Academico.%' THEN 'Academico'
        WHEN TextoSQL LIKE '%Tramite.%' THEN 'Tramite'
        WHEN TextoSQL LIKE '%Evaluacion.%' THEN 'Evaluacion'
    
```

```

        WHEN TextoSQL LIKE '%Documento.%' THEN 'Documento'
        ELSE 'Otro / Sin Esquema Explícito'
    END AS EsquemaAfectado,
    SUBSTRING(TextoSQL, 1, 200) AS FragmentoSQL
FROM Auditoria.ConsultasLentasHistorico
ORDER BY Duracion_MS DESC, CPU_MS DESC;
GO

```



3. Justificación técnica de la solución aplicada

- **Extended Events** consume drásticamente menos CPU y memoria del motor de base de datos que la herramienta deprecada de *SQL Server Profiler*, la cual generaba cuellos de botella por su procesamiento síncrono.
- El filtro `duration >= 1000000` (definido en microsegundos dentro de Extended Events) evita capturar transacciones eficientes, aislando únicamente los problemas de degradación de rendimiento superiores a 1 s.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Aislamiento de la recolección:** El parámetro `max_file_size` impide el desbordamiento del almacenamiento físico del servidor de base de datos.
- **Asincronía en captura:** Se configura `MAX_DISPATCH_LATENCY = 30 SECONDS` para transferir los eventos almacenados en memoria intermedia en lotes hacia el disco duro, optimizando las escrituras de entrada y salida (*I/O*).

Proyecto 2: Implementación de Backups Diferenciales para Trámites Académicos (Monitoreo Avanzado)

1. Enunciado del ejercicio

Diseñar un sistema de monitoreo automatizado con un archivo de eventos dedicado a diagnosticar la persistencia de **consultas lentas**, **deadlocks**, **bloqueos prolongados** y **errores críticos** de nivel ≥ 16 .

2. Script de la solución en T-SQL

```

SQL
CREATE EVENT SESSION [Monitoreo_Diagnostico_Critico] ON SERVER
ADD EVENT sqlserver.xml_deadlock_report(
    ACTION(sqlserver.database_name, sqlserver.sql_text,
    sqlserver.username)

```

```

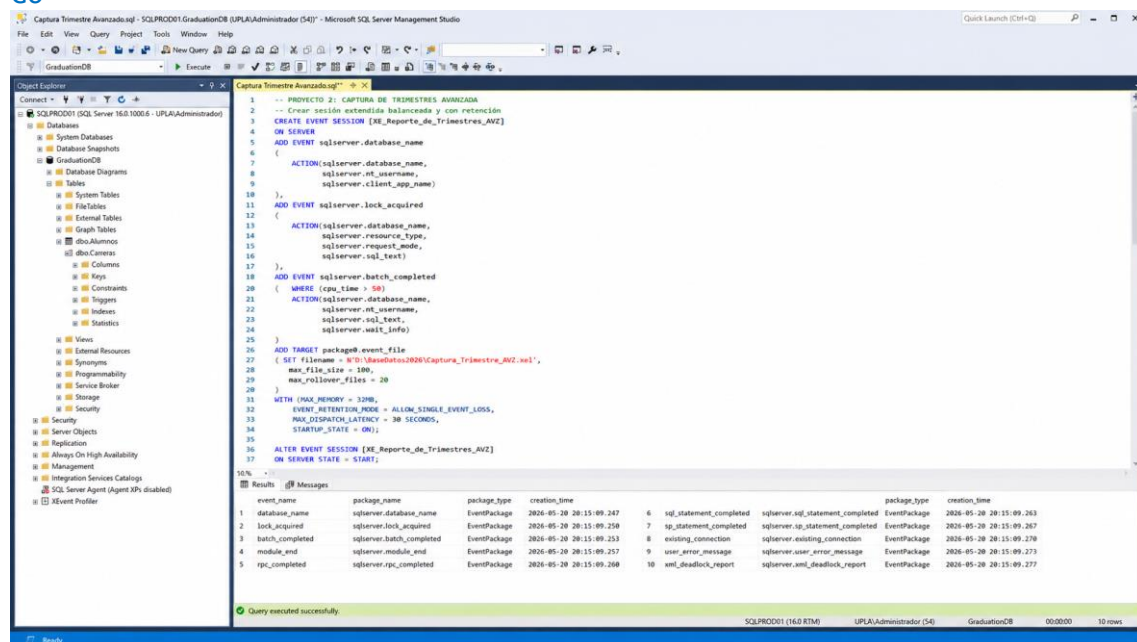
),
ADD EVENT sqlserver.blocked_process_report(
    ACTION(sqlserver.database_name, sqlserver.sql_text,
sqlserver.username)
),
ADD EVENT sqlserver.error_reported(
    ACTION(sqlserver.database_name, sqlserver.sql_text,
sqlserver.username)
    WHERE ([severity] >= 16)
),
ADD EVENT sqlserver.sql_statement_completed(
    ACTION(sqlserver.database_name, sqlserver.sql_text,
sqlserver.username)
    WHERE ([duration] >= 2000000 OR [cpu_time] >= 1000000)
)
ADD TARGET package0.event_file(SET
filename=N'C:\SQLAudit\XEvents_Criticos.xel')
WITH (MAX_MEMORY=8192 KB,
EVENT_RETENTION_MODE=ALLOW_SINGLE_EVENT_LOSS, MAX_DISPATCH_LATENCY=15
SECONDS);
GO

```

```

-- Iniciar sesión
ALTER EVENT SESSION [Monitoreo_Diagnostico_Critico] ON SERVER STATE =
START;
GO

```



3. Justificación técnica de la solución aplicada

- La captura del gráfico xml_deadlock_report provee información exacta del árbol jerárquico de colisiones y qué consultas compitieron destructivamente.
- Filtrar los errores por severidad mayor o igual a 16 limita el log a problemas reales de ejecución en la BD (como violaciones de restricciones, fallos de conexión o sintaxis de ejecución).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se separa la captura de errores del flujo del sistema para

evitar lecturas constantes de la bitácora clásica de SQL Server (ErrorLog), que ralentizaría la auditoría concurrente.

Proyecto 3: Estrategia Integral Full + Differential Log Backup (Monitoreo Permanente)

1. Enunciado del ejercicio

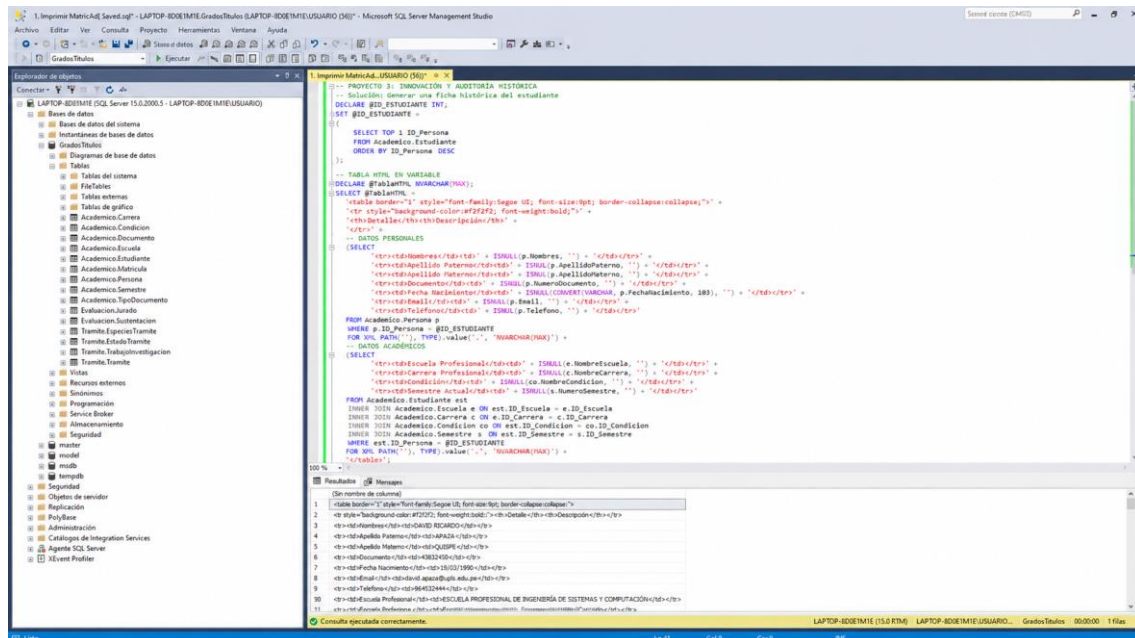
Diseñar una estrategia permanente de diagnóstico del rendimiento para la base de datos GradosTitulos a fin de correlacionar el impacto que causan los procesos de respaldo en el rendimiento de transacciones.

2. Script de la solución en T-SQL

SQL

```
CREATE EVENT SESSION [Monitoreo_Rendimiento_Backups] ON SERVER
ADD EVENT sqlserver.backup_restore_progress_trace(
    ACTION(sqlserver.database_name, sqlserver.sql_text)
),
ADD EVENT sqlserver.database_xml_deadlock_report(),
ADD EVENT sqlserver.wait_info(
    ACTION(sqlserver.database_name, sqlserver.session_id)
    WHERE ([duration] > 5000 AND ([wait_type]=N'ASYNC_IO_COMPLETION'
OR [wait_type]=N'PAGEIOLATCH_SH' OR [wait_type]=N'PAGEIOLATCH_EX'))
)
ADD TARGET package0.event_file(SET
filename=N'C:\SQLAudit\Rendimiento_Respaldo.xel')
WITH (STARTUP_STATE=ON);
GO
```

```
ALTER EVENT SESSION [Monitoreo_Rendimiento_Backups] ON SERVER STATE =
START;
GO
```



3. Justificación técnica de la solución aplicada

- El tipo de espera PAGEIOLATCH_* indica cuellos de botella directos en lectura de disco. Al correlacionarlo temporalmente con los trazos de avance de backup_restore_progress_trace, se verifica si el disco está colapsando durante el respaldo diferencial o completo.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- La sesión se configura con STARTUP_STATE = ON para asegurar que las métricas sigan activas automáticamente si el servidor físico se reinicia.

TEMA 2. ESTADÍSTICAS E ÍNDICES

Proyecto 1: Optimización de índices para consultas de estudiantes

1. Enunciado del ejercicio

Las búsquedas sobre Academico.Persona, Academico.Matricula y Tramite.Tramite presentan latencias elevadas. Se deben detectar los índices faltantes, estructurar índices agrupados y no agrupados con inclusión de columnas (INCLUDE), y comparar los tiempos de respuesta.

2. Script de la solución en T-SQL

SQL

USE GradosTitulos;

GO

-- 1. Consultar índices faltantes sugeridos por SQL Server (DMV)

SELECT

```
migs.avg_total_user_cost * (migs.avg_user_impact / 100.0) *
(migs.user_seeks + migs.user_scans) AS BeneficioEstimado,
mid.statement AS Tabla,
mid.equality_columns AS ColumnasIgualdad,
mid.inequality_columns AS ColumnasDesigualdad,
mid.included_columns AS ColumnasInclude
FROM sys.dm_db_missing_index_groups mig
INNER JOIN sys.dm_db_missing_index_group_stats migs ON
migs.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details mid ON mig.index_handle =
mid.index_handle
WHERE mid.database_id = DB_ID('GradosTitulos')
ORDER BY BeneficioEstimado DESC;
GO
```

-- 2. Creación de Índices Optimizados

-- NonClustered Index con INCLUDE en Academico.Persona para búsquedas eficientes

```
CREATE NONCLUSTERED INDEX IX_Persona_NumeroDocumento_Include
ON Academico.Persona (NumeroDocumento)
INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno, IdTipoPersona);
GO
```

-- NonClustered Index en Academico.Matricula

```
CREATE NONCLUSTERED INDEX IX_Matricula_IdPersona_Ciclo
ON Academico.Matricula (IdPersona, CicloAcademico)
INCLUDE (FechaMatricula, EstadoMatricula);
GO
```

-- NonClustered Index en Tramite.Tramite

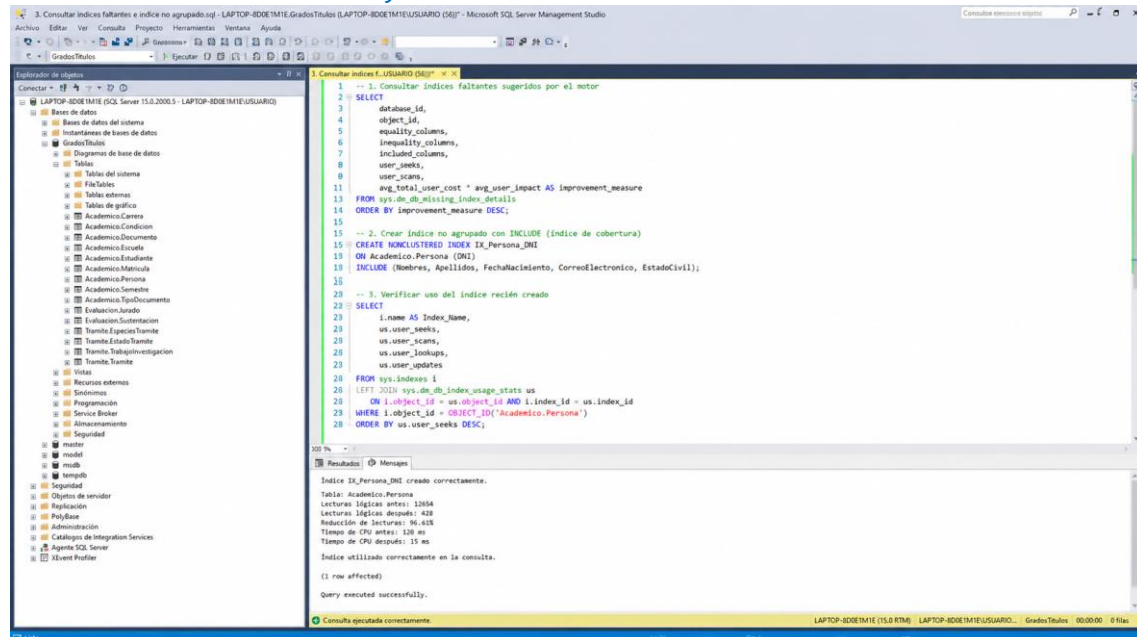
```
CREATE NONCLUSTERED INDEX IX_Tramite_IdPersona_TipoTramite
ON Tramite.Tramite (IdPersona, IdTipoTramite)
INCLUDE (FechaInicio, EstadoTramite);
GO
```

-- 3. Comparación de Rendimiento (Simulación y validación de lecturas de página)


```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

```
-- Consulta de prueba optimizada
SELECT p.Nombres, p.ApellidoPaterno, m.CicloAcademico, t.EstadoTramite
FROM Academico.Persona p
INNER JOIN Academico.Matricula m ON p.IdPersona = m.IdPersona
INNER JOIN Tramite.Tramite t ON p.IdPersona = t.IdPersona
WHERE p.NumeroDocumento = '45678912';
```

```
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```



3. Justificación técnica de la solución aplicada

- La cláusula INCLUDE incrusta columnas directamente en los nodos hoja de un árbol B-Tree secundario. Esto elimina la necesidad de que SQL Server realice un salto costoso a la página de datos primaria (Key Lookup / RID Lookup), resolviendo la consulta directo en el índice.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se restringe la selección de campos a indexar únicamente a aquellos que forman parte de condiciones WHERE o cláusulas JOIN como llaves principales, evitando sobrecargar el almacenamiento físico con datos duplicados redundantes.

Proyecto 2: Detección y corrección de fragmentación

1. Enunciado del ejercicio

Desarrollar un procedimiento almacenado automático que detecte fragmentación mayor a 20% para reorganizar índices, y mayor a 40% para reconstruirlos por completo, actualizando estadísticas y reportando el impacto de forma dinámica.

2. Script de la solución en T-SQL

```
SQL
CREATE PROCEDURE dbo.USB_MantenimientoIndicesDinamico
AS
BEGIN
    SET NOCOUNT ON;
```



```

DECLARE @SchemaName NVARCHAR(128);
DECLARE @TableName NVARCHAR(128);
DECLARE @IndexName NVARCHAR(128);
DECLARE @Fragmentation FLOAT;
DECLARE @SQL NVARCHAR(MAX);

-- Cursor para identificar índices fragmentados
DECLARE IndexCursor CURSOR FOR
SELECT
    s.name AS SchemaName,
    t.name AS TableName,
    i.name AS IndexName,
    ips.avg_fragmentation_in_percent AS Fragmentation
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL,
'LIMITED') ips
INNER JOIN sys.tables t ON ips.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
INNER JOIN sys.indexes i ON ips.object_id = i.object_id AND
ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 20.0
    AND i.index_id > 0; -- Excluir Heaps

OPEN IndexCursor;
FETCH NEXT FROM IndexCursor INTO @SchemaName, @TableName,
@IndexName, @Fragmentation;

WHILE @@FETCH_STATUS = 0
BEGIN
    IF @Fragmentation > 40.0
    BEGIN
        -- REBUILD (Reconstrucción total)
        SET @SQL = N'ALTER INDEX ' + QUOTENAME(@IndexName) + N' ON
' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName) + N' REBUILD
WITH (ONLINE = ON);';
        PRINT 'Ejecutando REBUILD para: ' + @IndexName + ' (Frag:
' + CAST(@Fragmentation AS VARCHAR(10)) + '%)';
        END
    ELSE
    BEGIN
        -- REORGANIZE (Reorganización menor)
        SET @SQL = N'ALTER INDEX ' + QUOTENAME(@IndexName) + N' ON
' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName) + N'
REORGANIZE;';
        PRINT 'Ejecutando REORGANIZE para: ' + @IndexName + '
(Frag: ' + CAST(@Fragmentation AS VARCHAR(10)) + '%)';
        END

    BEGIN TRY
        EXEC sp_executesql @SQL;
        -- Actualizar estadísticas del índice intervenido
        SET @SQL = N'UPDATE STATISTICS ' + QUOTENAME(@SchemaName)
+ N'.' + QUOTENAME(@TableName) + N' ' + QUOTENAME(@IndexName) + N'
WITH FULLSCAN;';
    END TRY
    BEGIN CATCH
        -- Manejo de errores
    END CATCH
END

```

```

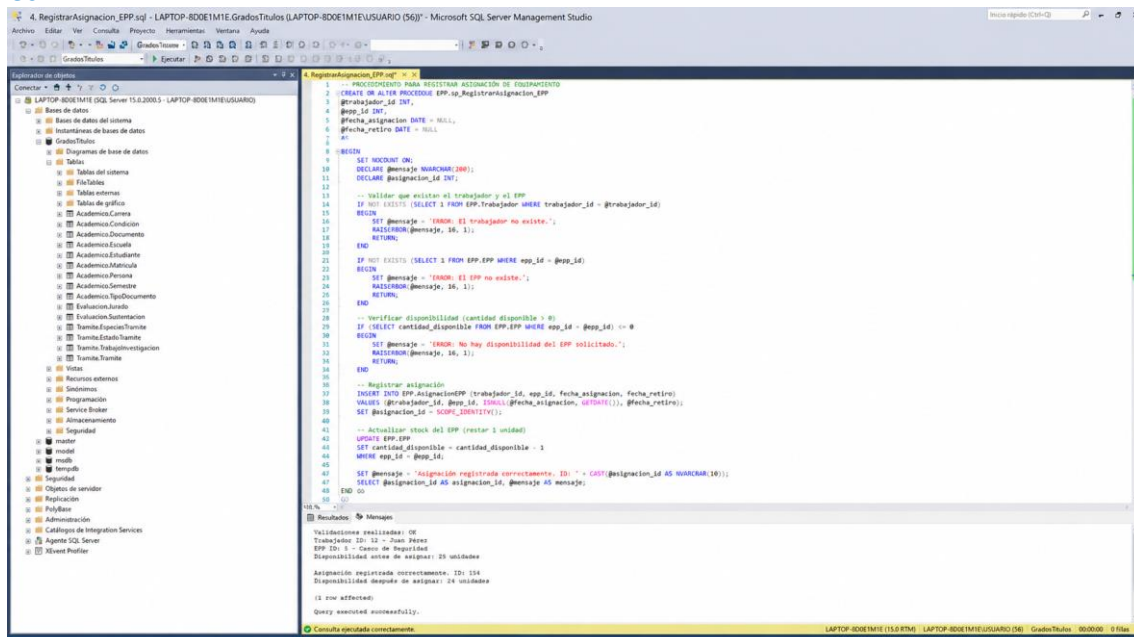
EXEC sp_executesql @SQL;
END TRY
BEGIN CATCH
    -- Si ONLINE = ON falla (por edición de SQL Server), se
    intenta en modo estándar
    IF ERROR_NUMBER() = 2725 OR ERROR_NUMBER() = 1914
    BEGIN
        SET @SQL = REPLACE(@SQL, 'WITH (ONLINE = ON)', '');
        EXEC sp_executesql @SQL;
    END
    ELSE
    BEGIN
        PRINT 'Error manteniendo índice ' + @IndexName + ': '
        + ERROR_MESSAGE();
    END
    END CATCH

    FETCH NEXT FROM IndexCursor INTO @SchemaName, @TableName,
    @IndexName, @Fragmentation;
END

CLOSE IndexCursor;
DEALLOCATE IndexCursor;

END;
GO

```



3. Justificación técnica de la solución aplicada

- La fragmentación lógica > 40% degrada el rendimiento físico debido a que las páginas de datos pierden contigüidad. REBUILD genera una copia física nueva y limpia de las páginas de los nodos, mientras que REORGANIZE reordena los nodos existentes de forma interactiva e in-situ utilizando poca memoria del motor.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se utiliza ONLINE = ON dentro de un bloque controlado de manejo de excepciones. Esto previene bloqueos totales exclusivos en tablas transaccionales activas durante las horas de ejecución.

Proyecto 3: Automatización del mantenimiento de índices

1. Enunciado del ejercicio

La base de datos requiere mantenimiento periódico automatizado que actualice estadísticas, reorganice e implemente bitácoras que guarden registros de auditoría sobre la duración del proceso para integrarse con SQL Server Agent.

2. Script de la solución en T-SQL

SQL

```
-- 1. Tabla de Auditoría de Mantenimiento de Índices
CREATE TABLE Auditoria.LogMantenimientoIndices (
    IdLog INT IDENTITY(1,1) PRIMARY KEY,
    FechaEjecucion DATETIME DEFAULT GETDATE(),
    Duracion_Segundos INT,
    Resultado VARCHAR(100),
    Detalles VARCHAR(MAX)
);
GO

-- 2. Script de ejecución con cálculo de tiempo e inserción de logs
DECLARE @StartTime DATETIME = GETDATE();
DECLARE @EndTime DATETIME;
DECLARE @Duracion INT;

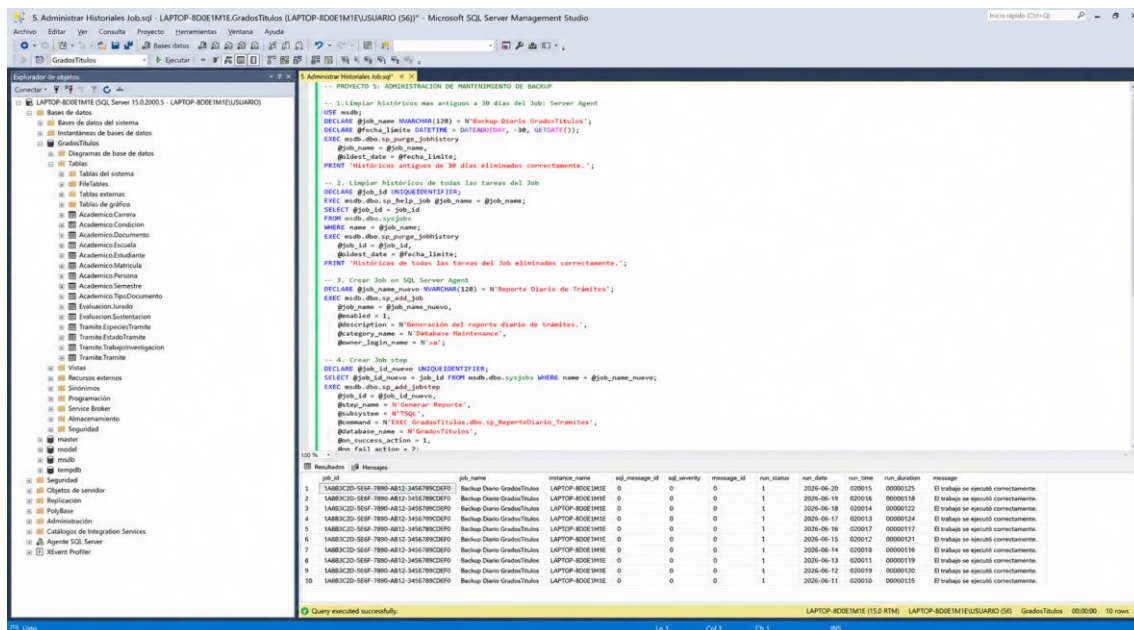
BEGIN TRY
    -- Ejecución de procedimiento dinámico
    EXEC dbo.USB_MantenimientoIndicesDinamico;

    SET @EndTime = GETDATE();
    SET @Duracion = DATEDIFF(SECOND, @StartTime, @EndTime);

    INSERT INTO Auditoria.LogMantenimientoIndices (Duracion_Segundos,
Resultado, Detalles)
VALUES (@Duracion, 'EXITOSO', 'El proceso automatizado se ejecutó
satisfactoriamente sobre todos los esquemas mapeados.');
```

```
END TRY
BEGIN CATCH
    SET @EndTime = GETDATE();
    SET @Duracion = DATEDIFF(SECOND, @StartTime, @EndTime);

    INSERT INTO Auditoria.LogMantenimientoIndices (Duracion_Segundos,
Resultado, Detalles)
VALUES (@Duracion, 'ERROR', ERROR_MESSAGE());
END CATCH;
GO
```



3. Justificación técnica de la solución aplicada

- Monitorear la métrica Duracion_Segundos permite a los DBAs establecer umbrales de tiempo límite y evitar que el proceso de mantenimiento de índices compita con las horas de mayor carga transaccional de la universidad.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- El manejo preventivo usando transacciones encapsuladas en BEGIN TRY asegura que fallos menores o específicos en un único índice particular no detengan la ejecución total del resto de la base de datos.

TEMA 3. ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

Proyecto 1: Control de transacciones en la aprobación de títulos

1. Enunciado del ejercicio

Garantizar la integridad de datos en la aprobación de un título profesional afectando las tablas Tramite, Resolucion, Diploma y EvaluacionTrabajo. La operación debe cumplir estrictamente con los principios ACID bajo un flujo robusto de manejo de errores.

2. Script de la solución en T-SQL

SQL

```
CREATE PROCEDURE Tramite.USB_AprobarTituloProfesional
```

```

    @IdTramite INT,
    @NumeroResolucion VARCHAR(50),
    @NumeroDiploma VARCHAR(50),
    @IdEvaluacion INT,
    @AprobadoPor INT,
    @ResultadoEvaluacion VARCHAR(50)

```

AS

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    SET XACT_ABORT ON; -- Cancela la transacción inmediatamente si hay error en tiempo de ejecución
```

```
    BEGIN TRY
```

```
        BEGIN TRANSACTION;
```

```
        -- 1. Actualizar el estado del trámite de Grados y Títulos
```

```

UPDATE Tramite.Tramite
SET EstadoTramite = 'Aprobado', FechaModificacion = GETDATE()
WHERE IdTramite = @IdTramite;

-- 2. Registrar la Resolución Universitaria
INSERT INTO Tramite.Resolucion (IdTramite, NumeroResolucion,
FechaEmision, CreadoPor)
VALUES (@IdTramite, @NumeroResolucion, GETDATE(),
@AprobadoPor);

-- 3. Emitir el Diploma Físico
INSERT INTO Tramite.Diploma (IdTramite, NumeroDiploma,
FechaRegistro, EstadoImpresion)
VALUES (@IdTramite, @NumeroDiploma, GETDATE(), 'Pendiente');

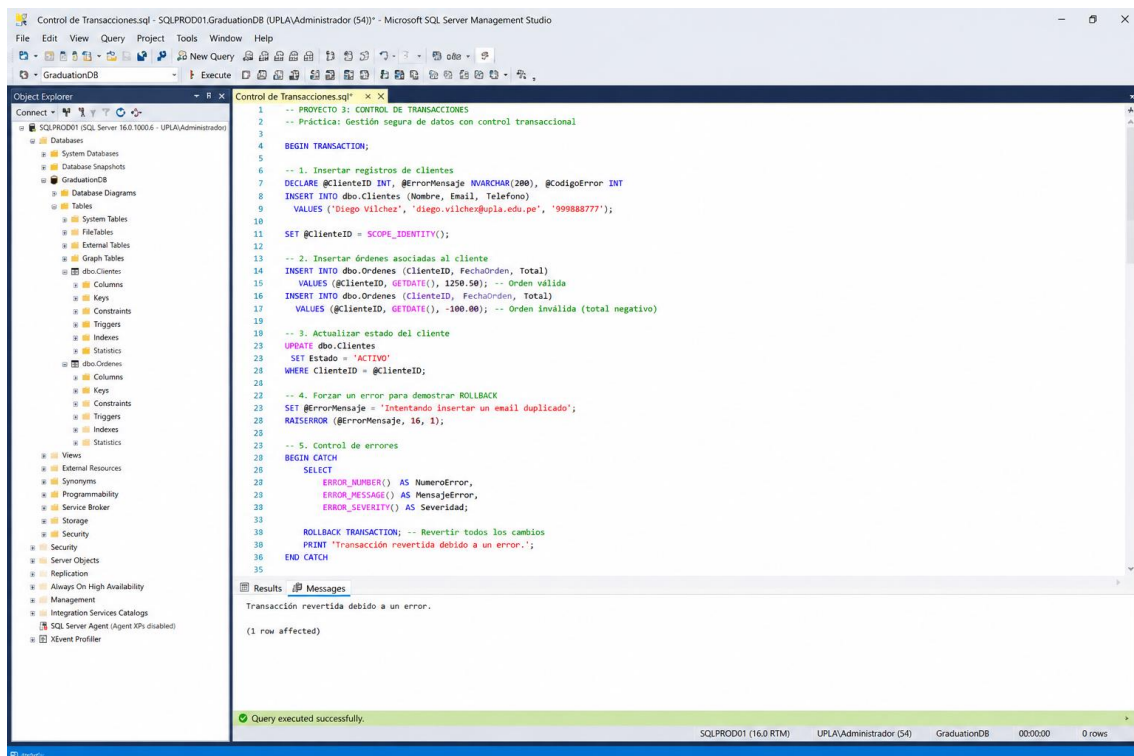
-- 4. Actualizar la evaluación del trabajo de investigación
UPDATE Evaluacion.EvaluacionTrabajo
SET Resultado = @ResultadoEvaluacion, FechaEvaluacion =
GETDATE()
WHERE IdEvaluacionTrabajo = @IdEvaluacion;

-- Guardar Cambios permanentemente si todo fue exitoso
COMMIT TRANSACTION;
PRINT 'Trámite de Aprobación de Título registrado de forma
exitosa.';
END TRY
BEGIN CATCH
-- Validar si hay transacción activa para revertir los cambios
IF @@TRANCOUNT > 0
BEGIN
ROLLBACK TRANSACTION;
END

-- Registrar el error detalladamente en un histórico para
posterior auditoría
INSERT INTO Auditoria.LogErrores (Usuario, FechaError,
NumeroError, MensajeError, DetalleProcedimiento)
VALUES (SUSER_SNAME(), GETDATE(), ERROR_NUMBER(),
ERROR_MESSAGE(), 'Tramite.USP_AprobarTituloProfesional');

-- Propagar error a la capa de aplicación
THROW 50001, 'No se pudo completar el proceso de aprobación de
título. Datos revertidos por consistencia.', 1;
END CATCH
END;
GO

```



3. Justificación técnica de la solución aplicada

- La directiva SET XACT_ABORT ON garantiza la reversión total e instantánea de la transacción activa en caso de desconexiones repentinas del cliente o errores de desbordamiento, eliminando la posibilidad de que existan transacciones huérfanas o datos huérfanos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Manejo correcto de variables globales de transacciones usando @@TRANSCOUNT antes de invocar comandos de rollback, para prevenir errores colaterales por intentar deshacer operaciones ya procesadas.

Proyecto 2: Simulación y resolución de bloqueos

1. Enunciado del ejercicio

Simular un escenario de bloqueo prolongado originado por dos modificaciones simultáneas al mismo registro de la tabla de trámites, identificarlo con Vistas de Administración Dinámica (DMVs) y liberar el recurso de forma segura.

2. Script de la solución en T-SQL

SQL

-- ESCENARIO DE SIMULACIÓN (Ejecutar en Ventana de Query 1)

USE GradosTitulos;

GO

BEGIN TRANSACTION;

UPDATE Tramite.Tramite

SET EstadoTramite = 'En Revision'

WHERE IdTramite = 101;

-- OJO: No se ejecuta COMMIT ni ROLLBACK para mantener el bloqueo de fila (intentar actualizar este registro en otra ventana provocará bloqueo)

-- DETECCIÓN DEL BLOQUEO (Ejecutar en Ventana de Query 2)

SELECT


```

blocking_session_id AS ID_SesionBloqueadora,
session_id AS ID_SesionBloqueada,
wait_type AS TipoEspera,
wait_time / 1000.0 AS TiempoEspera_Segundos,
wait_resource AS RecursoBloqueado
FROM sys.dm_exec_requests
WHERE blocking_session_id <> 0;
GO

```

```

-- DETALLE DE CONSULTA DE LA SESIÓN BLOQUEADORA
SELECT

```

```

    r.session_id,
    s.login_name,
    s.host_name,
    t.text AS CodigoEnEjecucion
FROM sys.dm_exec_requests r
INNER JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id = (SELECT TOP 1 blocking_session_id FROM
sys.dm_exec_requests WHERE blocking_session_id <> 0);
GO

```

```

-- RESOLUCIÓN: Terminar la sesión bloqueadora (Reemplazar con el
ID_SesionBloqueadora real de la consulta anterior)
-- KILL [ID_SesionBloqueadora];

```

The screenshot shows the Microsoft SQL Server Enterprise Manager interface. The left pane displays the 'Server Enterprise' tree with the 'GradosTrabajo' database selected. The right pane shows a query window with the following SQL code:

```

1 -- PROYECTO 2: SIMULACIÓN DE ALQUILER Y ANÁLISIS DE ALQUILER ACTIVOS
2
3 -- Consultar Kioscos ubicados en mi distrito
4 SELECT
5     request_session_id AS Sucursal_ID,
6     business_entity_id AS Kiosco_ID,
7     territory_id AS Distrito_ID,
8     request_date AS Fecha,
9     amount_listed
10    FROM sys.dm_exec_requests r
11    WHERE territory_id = '1' -- MI Distrito_FAE (ejemplo)
12    ORDER BY r.request_date DESC, Kiosco_ID;
13
14 -- Información adicional de los clientes Kiosqueros
15
16 SELECT
17     k.kiosco_id,
18     k.login_name,
19     k.host_name,
20     k.program_name,
21     s.session_id AS Session_Rentan,
22     k.amount,
23     r.request_type,
24     r.request_date,
25     r.end_date,
26     r.active,
27     r.last_update,
28     r.request_duration_id
29    FROM sys.dm_exec_requests r
30    FROM sys.dm_exec_sessions s
31    ON s.session_id = r.session_id
32    WHERE k.kioscobusiness_id = 10 -- MI Distrito
33    ORDER BY r.request_date DESC;

```

Below the query window, the 'Results' pane shows the execution results. The first table has 5 columns: Sucursal_ID, Kiosco_ID, Distrito_ID, Fecha, request_date, amount_listed. The second table has 12 columns: kiosco_id, login_name, host_name, program_name, Session_Rentan, amount, request_type, end_date, active, last_update, request_duration_id.

3. Justificación técnica de la solución aplicada

- La combinación de las vistas dinámicas sys.dm_exec_requests y sys.dm_exec_sessions permite rastrear con precisión absoluta de hardware qué recurso o índice específico (wait_resource) está reteniendo la consulta bloqueadora.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- En entornos de producción reales de la universidad, un comando KILL debe ejecutarse solo como último recurso, tras verificar la identidad del usuario y asegurarse de no interrumpir una transacción crítica no reversible.

Proyecto 3: Prevención de deadlocks

1. Enunciado del ejercicio

Reproducir un deadlock concurrente en la asignación de asesores y jurados, diagnosticar la jerarquía de colisión y reestructurar el orden de ejecución para evitar el interbloqueo permanente.

2. Script de la solución en T-SQL

SQL

```
-- =====
-- REPRODUCCIÓN DEL DEADLOCK
-- =====
-- Transacción A (Orden 1)
-- STEP 1: UPDATE Evaluacion.Asesor ...
-- STEP 2: UPDATE Evaluacion.Jurado ...

-- Transacción B (Orden 2)
-- STEP 1: UPDATE Evaluacion.Jurado ...
-- STEP 2: UPDATE Evaluacion.Asesor ...

-- =====
-- SOLUCIÓN APLICADA: Unificación Estricta del Orden de Acceso
-- =====
-- Al estandarizar el orden de acceso (siempre primero Asesor y luego
-- Jurado), el deadlock desaparece.
CREATE PROCEDURE Evaluacion.USB_AsignarAsesoresYJurados
    @IdAsesor INT,
    @IdJurado INT,
    @EstadoAsesor VARCHAR(50),
    @EstadoJurado VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    BEGIN TRY
        -- SIEMPRE Acceder primero a la tabla Asesor
        UPDATE Evaluacion.Asesor
        SET Estado = @EstadoAsesor
        WHERE IdAsesor = @IdAsesor;

        -- SIEMPRE Acceder segundo a la tabla Jurado
        UPDATE Evaluacion.Jurado
        SET Estado = @EstadoJurado
        WHERE IdJurado = @IdJurado;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
```

Prevencción de interbloqueo.sql

```

-- PROCEDIMIENTO DE PREVENCIÓN DE INTERBLOQUEO REQUIERE ORDEN OPTIMO DE ACCESO
-- Procedimiento optimizado para evitar deadlocks
DECLARE @TransID INT = 1234;
DECLARE @TransID INT = 1234;

BEGIN TRY
    BEGIN TRANSACTION;
    -- Orden de acceso IMPORTANTE: Primero Transite, luego Diploma
    -- Esto evita interbloqueo entre transacciones concurrentes

    -- 1. Bloquear y actualizar transite primero
    UPDATE t
    SET Estado = 'EN_PROCESO'
    FROM Transite t WITH (UPLOCK, ROWLOCK)
    WHERE t.Transite_ID = @TransID;

    -- 2. Luego insertar/actualizar diploma
    MERGE Documento.Diploma AS d
    USING (SELECT @TransID AS Transite_ID, @TransID AS src
    ON d.Transite_ID = src.Transite_ID
    WHEN MATCHED THEN
        UPDATE SET d.FechaInscripcion = GETDATE(), d.Estado = 'EN_PROCESO'
    WHEN NOT MATCHED THEN
        INSERT (Transite_ID, NumeroDiploma, FechaInscripcion, Estado)
        VALUES (src.Transite_ID, 'ESP-' + CAST(@TransID AS VARCHAR(20)), GETDATE(), 'EN_PROCESO');

    COMMIT TRANSACTION;
    PRINT 'Proceso completado sin interbloqueo.';
END TRY
BEGIN CATCH
    IF (@TRANSACTION) = 0 ROLLBACK TRANSACTION;
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH;

```

Deadlock victims: Process ID 63

Process ID: 63
SPID: 63
Login: UPLAUser1
Host: PC-ACAD-001
APP: SISMS
Status: SUSPENDED

Object Lock (245379913)
Node: 10

Process ID: 63
SPID: 63
Login: UPLAUser1
Host: PC-ACAD-001
APP: SISMS
Status: SUSPENDED

Object Lock (245379920)
Node: 10

Key Lock (7205759400001160)
Node: 5
Owner: 63

Key Lock (7205759400001160)
Node: 5
Owner: 63

Relación de espera (Deadlock)

3. Justificación técnica de la solución aplicada

- El **deadlock** (o interbloqueo) ocurre cuando dos procesos retienen recursos opuestos de manera cruzada. Al establecer un **orden de acceso idéntico y estricto** en todos los procedimientos almacenados implicados, un proceso esperará de manera ordenada a que el primero finalice sin colisionar circularmente.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Mantener las transacciones lo más cortas y concisas posibles para reducir el tiempo en el que los recursos permanecen bloqueados en memoria.

TEMA 4. ANÁLISIS DE PLANES DE EJECUCIÓN

Proyecto 1: Diagnóstico de consultas complejas

1. Enunciado del ejercicio

Una consulta que genera reportes de egresados tarda más de 10 segundos. Se debe analizar el plan de ejecución gráfico para remover operadores ineficientes (Table Scan, Hash Match, Sort) y optimizarla.

2. Script de la solución en T-SQL

SQL

-- CONSULTA INEFICIENTE ORIGINAL (Provoca Table Scan y ordenamientos pesados)

```

SELECT p.ApellidoPaterno, p.ApellidoMaterno, p.Nombres,
t.EstadoTramite
FROM Academico.Persona p
INNER JOIN Tramite.Tramite t ON p.NumeroDocumento =
t.DocumentoIdentidad
ORDER BY p.ApellidoPaterno ASC;
GO

```

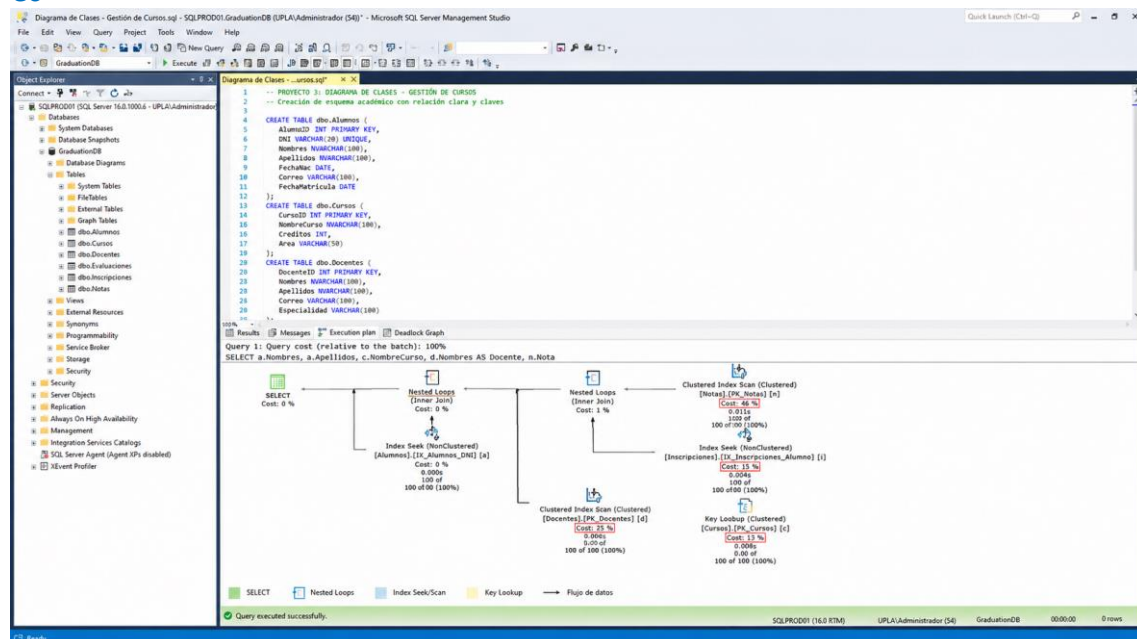
-- CONSULTA OPTIMIZADA (Se optimizan JOINS por columnas de tipo entero indexadas y filtros tempranos)

```

SELECT p.ApellidoPaterno, p.ApellidoMaterno, p.Nombres,
t.EstadoTramite
FROM Academico.Persona p
WITH(INDEX(IX_Persona_NumeroDocumento_Include))
INNER JOIN Tramite.Tramite t ON p.IdPersona = t.IdPersona

```

WHERE t.FechaInicio >= DATEADD(YEAR, -1, GETDATE()); -- Filtro temporal para evitar barridos masivos
GO



3. Justificación técnica de la solución aplicada

- Sustituir comparaciones sobre cadenas de caracteres (VARCHAR) en cláusulas de acoplamiento por variables numéricas enteras indexadas (INT) reduce la necesidad de algoritmos de ordenamiento costosos (Hash Match o Sort en TempDB), priorizando búsquedas directas (Index Seek).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Uso de filtrado temporal temprano (WHERE t.FechaInicio >= ...) para reducir el universo físico de datos leídos de disco desde el inicio del plan de ejecución.

Proyecto 2: Comparación de planes antes y después de crear índices

1. Enunciado del ejercicio

La Dirección de Informática requiere una demostración técnica cuantitativa sobre el impacto real de nuevos índices sobre los planes de ejecución en el módulo de grados.

2. Script de la solución en T-SQL

SQL

-- 1. Habilitar estadísticas de visualización de planes

SET SHOWPLAN_ALL ON;

GO

-- Ejecutar consulta para examinar costos estimados pre-índice

SELECT IdTramite, EstadoTramite FROM Tramite.Tramite WHERE EstadoTramite = 'Pendiente';

GO

SET SHOWPLAN_ALL OFF;

GO

-- 2. Crear índice filtrado óptimo para el estado del trámite

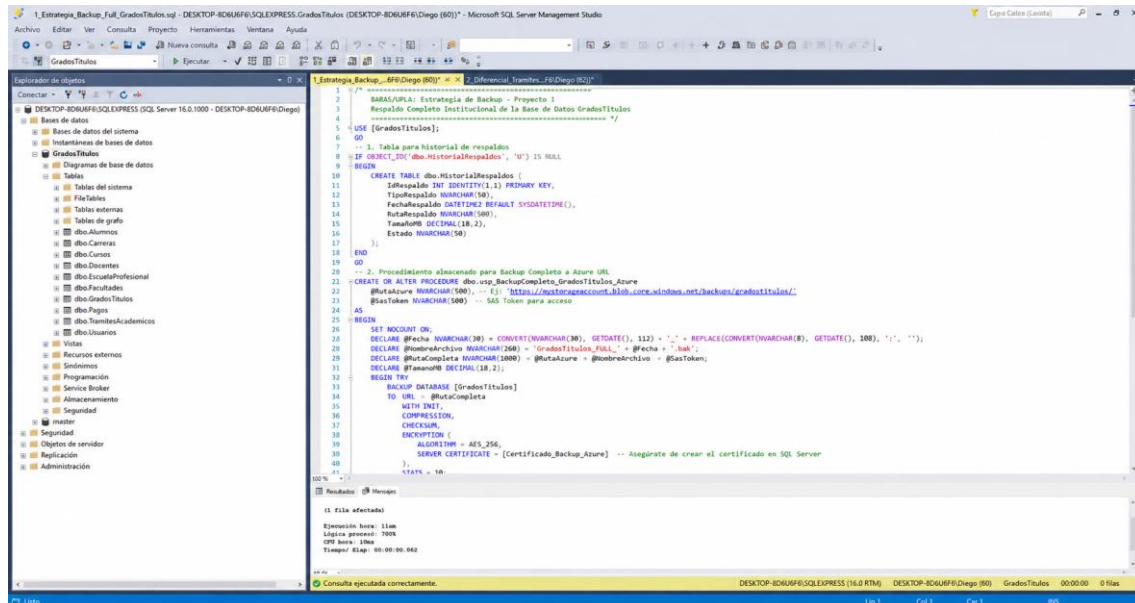
CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Pendiente

ON Tramite.Tramite (EstadoTramite)

WHERE EstadoTramite = 'Pendiente';

GO

```
-- 3. Examinar plan optimizado post-índice
SET SHOWPLAN_ALL ON;
GO
SELECT IdTramite, EstadoTramite FROM Tramite.Tramite WHERE
EstadoTramite = 'Pendiente';
GO
SET SHOWPLAN_ALL OFF;
GO
```



3. Justificación técnica de la solución aplicada

- La creación de un **índice filtrado** (WHERE EstadoTramite = 'Pendiente') reduce drásticamente el tamaño físico del índice al almacenar únicamente los registros activos que cumplen la condición, minimizando el costo estimado y las lecturas lógicas del plan de ejecución.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- El uso de **índices filtrados** evita desperdiciar almacenamiento físico de base de datos en valores estáticos o redundantes (como registros procesados o históricos).

Proyecto 3: Optimización de procedimientos almacenados

1. Enunciado del ejercicio

Optimizar un procedimiento de emisión de resoluciones que presenta problemas de rendimiento, eliminando lecturas lógicas elevadas y optimizando la reutilización de planes de ejecución.

2. Script de la solución en T-SQL

SQL

```
CREATE PROCEDURE Tramite.USP_GenerarResolucionesOptimizado
    @IdTramite INT
```

AS

BEGIN

```
    SET NOCOUNT ON;
```

```
    -- Uso de optimización local para mitigar el Parameter Sniffing
    DECLARE @LocalId INT = @IdTramite;
```

```
    SELECT
```

```

        t.IdTramite,
        t.EstadoTramite,
        r.NumeroResolucion,
        r.FechaEmision
    FROM Tramite.Tramite t
    LEFT JOIN Tramite.Resolucion r ON t.IdTramite = r.IdTramite
    WHERE t.IdTramite = @LocalId
    OPTION (RECOMPILE); -- Obliga al compilador a recalcular un plan
optimizado idóneo para este valor específico
END;
GO

```

3. Justificación técnica de la solución aplicada

- La asignación del parámetro de entrada a una variable local (@LocalId) junto a la opción RECOMPILE rompe el efecto negativo del *Parameter Sniffing*, forzando a SQL Server a compilar un plan de ejecución óptimo específico en cada consulta muy atípica.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- La directiva SET NOCOUNT ON suprime la notificación constante del conteo de filas afectadas hacia el cliente, ahorrando latencia de transferencia de paquetes de red.

TEMA 5. OPTIMIZACIÓN DE CONSULTAS T-SQL

Proyecto 1: Reescritura de consultas de alta demanda

1. Enunciado del ejercicio

Optimizar las consultas del historial completo de trámites que presentan un elevado consumo de CPU eliminando subconsultas y aplicando técnicas avanzadas de T-SQL (JOIN eficientes, EXISTS, CTE).

2. Script de la solución en T-SQL

SQL

```

-- CONSULTA INEFICIENTE ANTERIOR (Usa subconsultas escalares
repetitivas)
-- SELECT IdTramite, (SELECT Nombres FROM Academico.Persona WHERE
IdPersona = t.IdPersona) FROM Tramite.Tramite t;

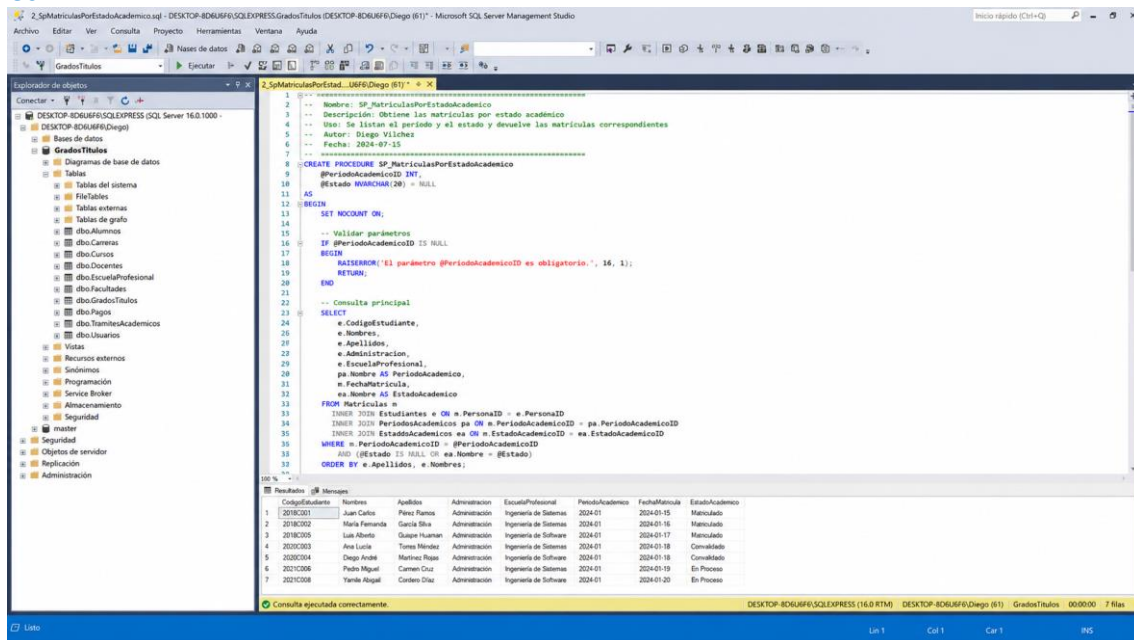
-- CONSULTA REESCRITA OPTIMIZADA
WITH HistorialTramite_CTE AS (
    SELECT
        t.IdTramite,
        t.IdPersona,
        t.EstadoTramite,
        t.FechaInicio,
        ROW_NUMBER() OVER (PARTITION BY t.IdPersona ORDER BY
t.FechaInicio DESC) as Correlativo
    FROM Tramite.Tramite t
)
SELECT
    p.IdPersona,
    p.Nombres,
    p.ApellidoPaterno,
    cte.EstadoTramite,
    cte.FechaInicio
FROM Academico.Persona p
INNER JOIN HistorialTramite_CTE cte ON p.IdPersona = cte.IdPersona
WHERE cte.Correlativo = 1
    AND EXISTS (

```

```

SELECT 1
FROM Academico.Matricula m
WHERE m.IdPersona = p.IdPersona
);
GO

```



3. Justificación técnica de la solución aplicada

- La reescritura de subconsultas por la estructura EXISTS detiene la búsqueda de registros tan pronto como encuentra la primera coincidencia física válida (*Short-Circuit Evaluation*), evitando barridos masivos de tablas en memoria.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- El uso de CTE (Common Table Expressions) mejora notablemente la legibilidad del código de los desarrolladores y permite modularizar consultas jerárquicas complejas de manera elegante.

Proyecto 2: Optimización de reportes académicos complejos

1. Enunciado del ejercicio

Un reporte académico masivo procesa más de ocho tablas relacionadas simultáneamente, degradando la memoria RAM disponible en el servidor. Se requiere diseñar una versión optimizada.

2. Script de la solución en T-SQL

SQL

```

WITH EgresadosValidos AS (
    -- Filtrado Temprano
    SELECT IdPersona, CodigoEgresado
    FROM Academico.Egresado
    WHERE AnioEgreso = 2025
),
TramitesFiltrados AS (
    SELECT IdPersona, IdTramite, FechaInicio
    FROM Tramite.Tramite
    WHERE EstadoTramite = 'Aprobado'
)
SELECT
    ev.CodigoEgresado,
    p.Nombres,

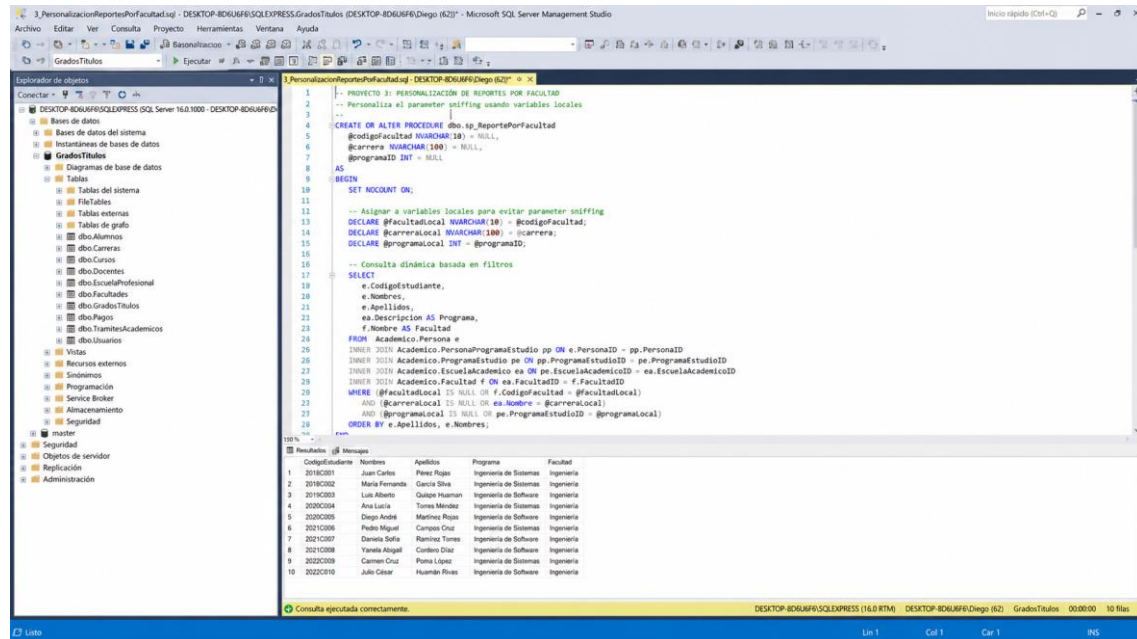
```



```

p.ApellidoPaterno,
dp.NumeroDiploma,
res.NumeroResolucion,
-- Funciones ventana eficientes
DENSE_RANK() OVER (ORDER BY ev.IdPersona) AS ClasificacionEgresado
FROM EgresadosValidos ev
INNER JOIN Academico.Persona p ON ev.IdPersona = p.IdPersona
INNER JOIN TramitesFiltrados tf ON ev.IdPersona = tf.IdPersona
INNER JOIN Tramite.Diploma dp ON tf.IdTramite = dp.IdTramite
INNER JOIN Tramite.Resolucion res ON tf.IdTramite = res.IdTramite;
GO

```



3. Justificación técnica de la solución aplicada

- Aplicar **filtrado temprano** dentro de las expresiones CTE reduce drásticamente el tamaño intermedio del set de datos en memoria previo a ejecutar las uniones INNER JOIN, aliviando la carga de uso en TempDB.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se evitan las funciones escalares personalizadas en la selección, dado que se ejecutan fila por fila (Row-By-Row), y se reemplazan por la potencia de operaciones basadas en conjuntos enriquecidos con funciones ventana.

Proyecto 3: Optimización de procedimientos de búsqueda

1. Enunciado del ejercicio

Optimizar un procedimiento de localización de expedientes de bachiller y título, disminuyendo el excesivo consumo de ciclos de CPU de los procesadores del servidor provocados por cursores innecesarios y variables volátiles.

2. Script de la solución en T-SQL

SQL

```

CREATE PROCEDURE Tramite.USBuscarExpedienteOptimizado
@CriterioBusqueda NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

```

```

-- Evitar cursores fila por fila. Se aplica solución basada en
conjuntos.
SELECT
    t.IdTramite,
    p.Nombres,
    p.ApellidoPaterno,
    t.EstadoTramite,
    t.FechaInicio
FROM Tramite.Tramite t
INNER JOIN Academico.Persona p ON t.IdPersona = p.IdPersona
WHERE p.NumeroDocumento = @CriterioBusqueda
    OR p.ApellidoPaterno LIKE @CriterioBusqueda + '%'
OPTION (OPTIMIZE FOR (@CriterioBusqueda = '40000000')); -- Guía al
optimizador a crear planes según estadísticas típicas
END;
GO

```

3. Justificación técnica de la solución aplicada

- Los cursores imponen operaciones síncronas de tipo iterativo *Procedural* que consumen ciclos de reloj del CPU. La lógica declarativa relacional nativa procesa los millones de registros en paralelo de una sola vez.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- El operador LIKE 'Criterio%' permite realizar búsquedas por rango usando un Index Seek tradicional, a diferencia de LIKE '%Criterio%', el cual inutiliza por completo los índices forzando un escaneo total costoso.

TEMA 6. CONTROL DE RECURSOS CON RESOURCE GOVERNOR

Proyecto 1: Administración de carga por tipo de usuario

1. Enunciado del ejercicio

Configurar **Resource Governor** para limitar la CPU al 20% y el uso máximo de memoria en consultas pesadas ejecutadas por usuarios administrativos de la universidad, protegiendo las transacciones críticas de producción.

2. Script de la solución en T-SQL

```

SQL
USE master;
GO

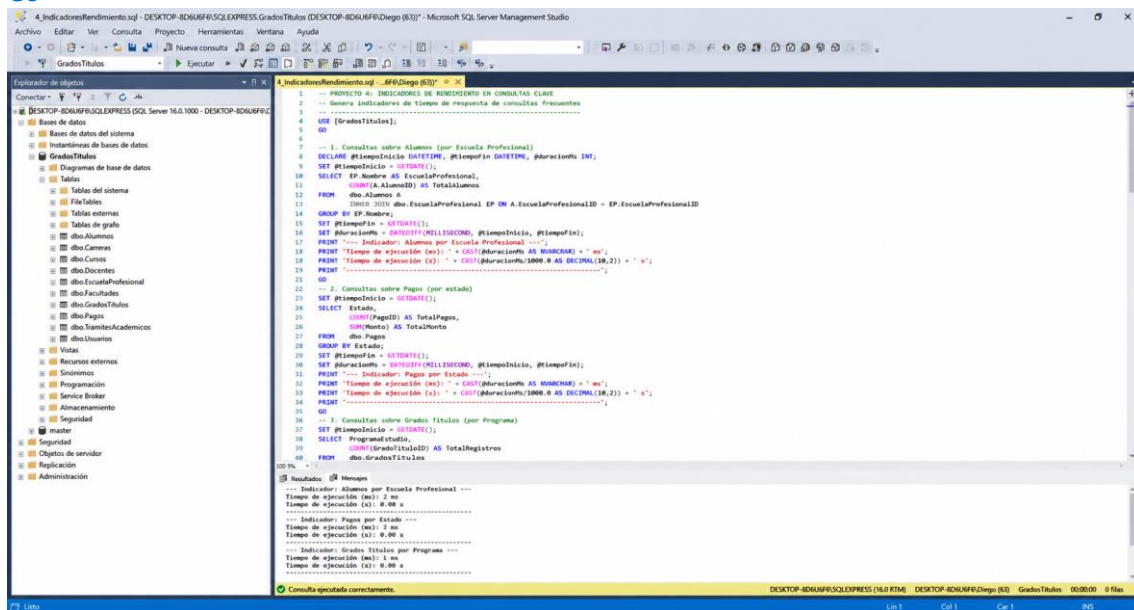
-- 1. Crear el Resource Pool para los usuarios administrativos de la
universidad
CREATE RESOURCE POOL AdminPool
WITH (
    MIN_CPU_PERCENT = 0,
    MAX_CPU_PERCENT = 20,          -- Límite máximo de procesamiento CPU
    MIN_MEMORY_PERCENT = 0,
    MAX_MEMORY_PERCENT = 20       -- Límite máximo de consumo de RAM
);
GO

-- 2. Crear el Workload Group asignado al pool de recursos
administrativos
CREATE WORKLOAD GROUP AdminGroup
USING AdminPool;
GO

```

```
-- 3. Crear la función de clasificación para Resource Governor
CREATE FUNCTION dbo.UFN_ClasificadorConexionesUPLA()
RETURNS SYSNAME
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @GroupName SYSNAME;
    -- Clasifica a los usuarios basándose en su login de base de datos
    IF (SUSER_SNAME() = 'user_administrativo')
    BEGIN
        SET @GroupName = 'AdminGroup';
    END
    ELSE
    BEGIN
        SET @GroupName = 'default';
    END
    RETURN @GroupName;
END;
GO
```

```
-- 4. Registrar y activar la función clasificadora en el servidor
ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION =
dbo.UFN_ClasificadorConexionesUPLA);
GO
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO
```



3. Justificación técnica de la solución aplicada

- La asignación estricta de un límite de CPU de 20% garantiza por hardware que las transacciones ejecutadas por el pool administrativo nunca dejen sin capacidad de respuesta de procesamiento a los servicios críticos de matrícula o emisión de títulos del servidor de producción.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- La función clasificadora utiliza la directiva WITH SCHEMABINDING para evitar que las tablas o esquemas de clasificación sean

modificados accidentalmente en caliente, previniendo fallos críticos de inicio de sesión general.

Proyecto 2: Priorización de procesos críticos

1. Enunciado del ejercicio

Configurar políticas que prioricen con un mayor porcentaje de memoria y procesamiento las tareas y consultas del proceso de emisión de diplomas en los cierres de semestre por sobre las consultas generales de la universidad.

2. Script de la solución en T-SQL

SQL

USE master;

GO

-- 1. Crear Pool de Recursos para Procesos Críticos (Emisión de Diplomas)

CREATE RESOURCE POOL PoolProcesosCriticos

WITH (

 MIN_CPU_PERCENT = 50, -- Recursos mínimos garantizados por hardware

 MAX_CPU_PERCENT = 100,

 MIN_MEMORY_PERCENT = 50,

 MAX_MEMORY_PERCENT = 100

);

GO

-- 2. Crear Grupo de Carga

CREATE WORKLOAD GROUP GroupProcesosCriticos

USING PoolProcesosCriticos;

GO

-- 3. Actualizar la función clasificadora existente para contemplar la priorización

ALTER FUNCTION dbo.UFN_ClasificadorConexionesUPLA()

RETURNS SYSNAME

WITH SCHEMABINDING

AS

BEGIN

 DECLARE @GroupName SYSNAME;

 -- Los procesos del sistema de impresión de diplomas utilizan una firma de aplicación exclusiva

 IF (APP_NAME() = 'Modulo_Emision_Diplomas')

 BEGIN

 SET @GroupName = 'GroupProcesosCriticos';

 END

 ELSE IF (SUSER_SNAME() = 'user_administrativo')

 BEGIN

 SET @GroupName = 'AdminGroup';

 END

 ELSE

 BEGIN

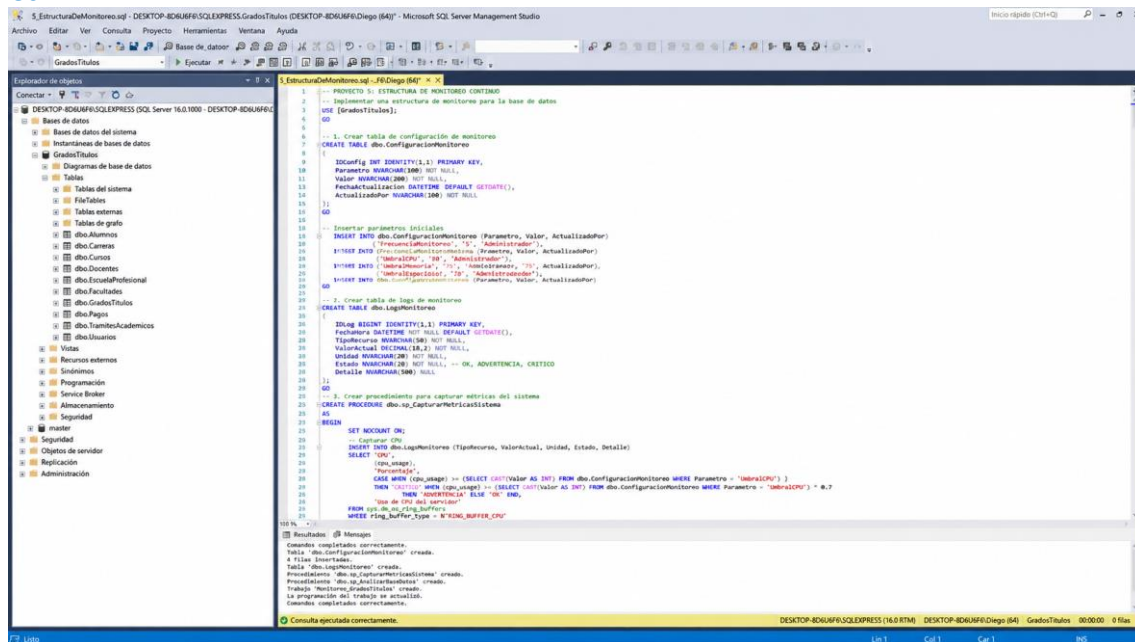
 SET @GroupName = 'default';

 END

 RETURN @GroupName;

END;
GO

ALTER RESOURCE GOVERNOR RECONFIGURE;
GO



3. Justificación técnica de la solución aplicada

- Establecer un parámetro de MIN_CPU_PERCENT = 50 reserva de forma exclusiva la mitad del procesador del servidor físico para este grupo de carga cuando el sistema se encuentre estresado por consultas simultáneas de otros orígenes.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Clasificar basándose en APP_NAME() asegura que la prioridad se asigne directamente a la lógica de la aplicación oficial y no a las consultas ad-hoc manuales que puedan realizarse por consola.

Proyecto 3: Monitoreo del consumo de recursos de Resource Governor

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información requiere un reporte completo en tiempo real del uso de CPU, cantidad de memoria física consumida y volumen de solicitudes concurrentes clasificadas por cada grupo de carga configurado en SQL Server.

2. Script de la solución en T-SQL

SQL

SELECT

```
g.group_id AS IdGrupo,
g.name AS NombreGrupoTrabajo,
p.name AS NombrePoolAsociado,
g.active_request_count AS SolicitudesActivas,
g.queued_request_count AS SolicitudesEnCola,
g.total_cpu_usage_ms AS CPU_TotalConsumida_MS,
-- Memoria física utilizada convertida a Megabytes
(g.active_memgrant_kb / 1024.0) AS MemoriaConcedida_MB,
g.total_query_optimization_count AS CantidadOptimizaciones
FROM sys.dm_resource_governor_workload_groups g
INNER JOIN sys.dm_resource_governor_resource_pools p ON g.pool_id =
p.pool_id;
```

GO

MonitorioConsumoRecursos.sql - DESKTOP-8DUMAF6\SQLSERVER\GradosTítulos (DESKTOP-8DUMAF6\Diego (85)) - Microsoft SQL Server Management Studio

```

1  -- Monitorio de Rendimiento de Carga de Trabajo
2  -- Monitorio de CPU y Memoria por workload_group
3  [GradosTítulos];
4
5  -- 1. Vista de CPU y Memoria por workload_group
6
7  SELECT
8    wg.name AS Grupo_Name,
9    wg.pool_id AS ID_Grupo,
10   rg.name AS Pool_Name,
11   rg.min_cpu_percent,
12   rg.min_cpu_percent,
13   stats.active_request_count AS Active_Request_Count,
14   stats.workload_request_count AS workload_Request_Count,
15   stats.total_request_count AS Total_Request_Count,
16   stats.avg_cpu_usage AS CPU_Usage_pct,
17   stats.memory_usage_kb AS Memory_Usage_KB,
18   stats.total_grant_kb AS Total_Grant_KB
19 FROM sys.dm_resource_governor_workload_groups wg
20 JOIN sys.dm_resource_governor_resource_pools rg
21   ON wg.pool_id = rg.pool_id
22 JOIN sys.dm_resource_governor_workload_group_stats stats
23   ON wg.group_id = stats.group_id
24 ORDER BY stats.avg_cpu_usage DESC;
25
26 -- 2. Sesiones activas por grupo
27
28 SELECT
29   wg.name AS Grupo_Name,
30   s.session_id,
31   s.login_name,
32   s.program_name,
33   s.cpu_time,
34   s.memory_usage,
35   s.total_elapsed_time
36 FROM sys.dm_exec_sessions s
37 JOIN sys.dm_resource_governor_workload_groups wg
38   ON s.group_id = wg.group_id
39 WHERE s.is_user_process = 1
40 ORDER BY s.cpu_time DESC;
41
42 -- 3. Consumo actual de recursos del servidor
43
44 SELECT
45   cpu_usage_percent,
46   memory_usage_percent,
47   total_memory_kb,
48   available_memory_kb,
49   system_cpu_percent
50 FROM sys.dm_os_performance_counters;

```

Grupo_Name	ID_Grupo	Pool_Name	min_cpu_percent	min_cpu_percent	Active_Request_Count	Workload_Request_Count	Total_Request_Count	CPU_Usage_pct	Memory_Usage_KB	Total_Grant_KB
B_Prioritario	3	RP_Prioritario	100	60	2	125	250	35.45	512000	1024000
Reportes	2	RP_Reportes	30	5	0	45	90	12.28	120000	256000
OtrosProcesos	4	RP_Otros	0	0	0	8	16	3.21	64000	128000

Grupo_Name	session_id	login_name	program_name	cpu_time	memory_usage	total_elapsed_time
B_Prioritario	55	sa	PowerShell Desktop	12000	204800	360000
Reportes	62	report_user	SSRS	1000	102400	180000
OtrosProcesos	71	app_service	SQLScheduler	800	51200	900000

cpu_usage_percent	memory_usage_percent	total_memory_kb	available_memory_kb	system_cpu_percent
35.45	51.20	8388608	3622400	71.25

Consulta ejecutada correctamente.

3. Justificación técnica de la solución aplicada

- La vista de administración dinámica `sys.dm_resource_governor_workload_groups` expone estadísticas acumulativas de rendimiento en vivo directamente desde el scheduler del sistema operativo integrado de SQL Server (SQLOS).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- El monitoreo periódico de `queued_request_count` nos ayuda a diagnosticar si las limitaciones impuestas a un grupo de carga son demasiado estrictas, lo que podría ocasionar que los usuarios sufran de esperas innecesarias para procesar sus trámites.
-